

Cálculo Teórico de Workers para Alta Concurrencia y Optimización

Especificaciones del Entorno

- **Plataforma:** Oracle Cloud Free Tier (Instancia ARM Ampere)
- **CPUs:** 4 Procesadores (Threads/Cores)
- **RAM Total:** 24 GB
- **Almacenamiento:** 100 GB HDD
- **Objetivo:** Tolerancia a miles de peticiones simultáneas sin OOM (Out Of Memory) ni contención crítica de CPU.

Análisis de Distribución de Memoria (RAM)

Para garantizar la estabilidad, debemos hacer un presupuesto estricto de memoria:

1. **Sistema Operativo y Stack Docker base:** ~2 GB.
2. **Servicios Auxiliares** (ChromaDB, OSRM, MQTT Mosquitto, FastAPI Ingress, Loggers): ~3.5 GB.
3. **Model-Up Service (Mistral 7B Q4_K_M):** ~7 GB cargados en RAM (incluye la memoria del modelo + un buffer modesto de KV cache para el contexto).
4. **Mistral Service (Qwen2.5 3B Q4_K_M):** ~4 GB cargados en RAM.
5. **14 Servicios Secundarios** (FastAPI I/O Bound): ~1.5 GB en total (~100MB por servicio).

Total de Memoria Comprometida: ~18 GB. **Margen de Seguridad (RAM Libre):** ~6 GB (Crítico para picos de procesamiento de requests, manejo de conexiones TCP concurrentes, y buffer de I/O en disco).

Análisis de Procesamiento (CPU)

Ambos servicios de LLM (`llama.cpp`) están configurados por defecto con `n_threads = 4`. Al tener solo 4 procesadores físicos/virtuales en la instancia, **ejecutar más de una inferencia en paralelo producirá una severa contención de hilos**, degradando el rendimiento de forma exponencial (Thrashing de CPU) e incrementando significativamente los tiempos de respuesta.

Cálculo y Asignación Óptima de Workers (Uvicorn)

1. Servicios de Inferencia (Model-Up y Mistral Service):

- **Workers Óptimos: 1 Worker** (`--workers 1`).
- *Justificación:* Al usar Uvicorn, cada "worker" clona el proceso completo. Si levantamos 2 workers en `model_up_service`, el consumo de RAM pasaría de 7 GB a 14 GB (superando nuestra RAM libre y causando un Out of Memory inmediato o forzando el uso de SWAP en disco HDD que destruiría el rendimiento).
- *Solución de Concurrencia (Cola implementada):* Para aceptar miles de peticiones y no rechazar conexiones, el único worker de Uvicorn maneja la red de forma asíncrona, encolando las peticiones en el *Event Loop*. Hemos configurado un `ThreadPoolExecutor(max_workers=1)` en Python para garantizar que **solo se procese exactamente 1 petición a la vez hacia el modelo**.

Las miles de peticiones adicionales esperarán pacíficamente de forma asíncrona en memoria consumiendo solo unos pocos kilobytes cada una, sin causar OOM.

2. Servicio Ingress y Microservicios Secundarios (I/O Bound):

- **Workers Óptimos: 1 a 2 Workers** (Ya están limitados o usan los defaults en sus Dockerfiles).
- *Justificación:* Estos servicios solo reciben peticiones HTTP, parsean el JSON, buscan en BD y envían la petición a Model-Up. Escalar estos servicios a 4 workers consumiría RAM adicional y no aumentaría el Throughput global de la aplicación, ya que el cuello de botella siempre será el tiempo de inferencia de la IA. Mantenerlos en 1 worker maximiza la RAM libre para el sistema.

Verificación de Flujo de Modelos

- Se ha revisado el código base de todos los microservicios secundarios (transporte, registro, ruta, venta, etc.).
- **Ninguno levanta modelos de HuggingFace de forma autónoma.**
- Todos apuntan correctamente por variable de entorno `MODEL_UP_URL` hacia `http://model_up_service:8040/infer`, delegando la carga computacional pesada y operando estrictamente como enrutadores lógicos ligeros.

Conclusión Arquitectónica

La combinación de **1 Uvicorn Worker + ThreadPoolExecutor(max_workers=1)** actúa como un sistema de *Cola en Memoria* (Memory-based Queue) altamente resiliente. Permite encolar peticiones de forma asíncrona hasta agotar el límite de file descriptors del OS (decenas de miles), manteniéndolas en espera y alimentándolas al modelo 1 a 1. Esto estabiliza completamente el uso de RAM (~18GB/24GB fijos) y el uso de CPU (100% eficiente pero no sobre-saturado), logrando la optimización requerida en la infraestructura de Oracle Cloud.